



**UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO**  
**FACULTAD DE CIENCIAS DE LA COMPUTACIÓN**  
**CARRERA INGENIERÍA EN SOFTWARE**  
**PROCESO DE SOFTWARE "A"**

**ARTEFACTO A12:**  
**Documento de Estándares de Codificación C# (.NET)**

**PROYECTO:**  
**Sistema de Reserva de Canchas Sintéticas**



**PRESENTADO POR:**  
**Calderón Saltos Joseph Alexander**  
**Herrera Barco Humberto Aldair**  
**Reinoso Vélez Eduardo David**  
**Silva Triviño John Jairo**

**FECHA:**  
**Miércoles, 8 de julio del 2026**

## 1. Introducción y objetivo

Este documento establece las convenciones de codificación obligatorias para el desarrollo del Sistema de Reserva de Canchas Sintéticas, una aplicación de escritorio construida en Windows Forms (C#) sobre .NET Framework 4.8, con SQL Server 2022 como motor de base de datos, siguiendo la arquitectura de cuatro capas definida en el artefacto A3 (Presentación, Negocio, Datos y Entidades).

El propósito de este estándar es que, al trabajar cuatro integrantes en paralelo sobre módulos distintos (clientes, reservas, pagos, canchas y usuarios), el código resultante sea legible, uniforme y fácil de integrar, independientemente de quién lo haya escrito. Se aplica desde el inicio de la fase de Construcción y Pruebas y es de cumplimiento obligatorio en revisiones de código (pull requests) antes de fusionar a la rama principal.

### Alcance

Aplica a todo el código C# del proyecto (proyectos de Presentación, Negocio, Datos y Entidades) y a los procedimientos almacenados de SQL Server definidos en el artefacto A11 (Script DDL).

Este documento se apoya y debe leerse junto con: A1 – Especificación de Requisitos, A3 – Arquitectura en Capas, A6 – Modelo Lógico, A7 – Modelo Físico y A11 – Script DDL.

## 2. Organización de la solución y los proyectos

La solución de Visual Studio refleja físicamente las cuatro capas del artefacto A3, separadas en proyectos independientes dentro de una misma solución. Esto evita el acoplamiento entre capas y obliga a que la comunicación entre ellas pase por interfaces públicas bien definidas.

```
SistemaCanchasSinteticas.sln
|
+-- SistemaCanchas.Presentacion/      (proyecto de inicio - Windows Forms)
|   +-- FrmLogin.cs
|   +-- FrmClientes.cs
|   +-- FrmReservas.cs
|   +-- FrmPagos.cs
|   +-- FrmCanchas.cs
|   +-- FrmUsuarios.cs
|   +-- Recursos/                     (iconos, imagenes)
|
+-- SistemaCanchas.Negocio/           (class library)
|   +-- UsuarioService.cs
|   +-- ClienteService.cs
|   +-- ReservaService.cs
|   +-- PagoService.cs
|   +-- CanchaService.cs
|   +-- Interfaces/
|       +-- IUserarioService.cs
|       +-- IReservaService.cs ...
|   +-- Excepciones/
|       +-- ReservaConflictoException.cs
|       +-- CredencialesInvalidasException.cs
|
+-- SistemaCanchas.Datos/             (class library)
```

```

|   +-- GestorConexion.cs           (login de dos fases: bootstrap -> real)
|   +-- UsuarioRepository.cs
|   +-- ClienteRepository.cs
|   +-- ReservaRepository.cs
|   +-- PagoRepository.cs
|   +-- CanchaRepository.cs
|
+-- SistemaCanchas.Entidades/      (class library - POCOs)
|   +-- Cliente.cs
|   +-- Reserva.cs
|   +-- Horario.cs
|   +-- Pago.cs
|   +-- Cancha.cs
|   +-- Usuario.cs
|   +-- Rol.cs
|
+-- SistemaCanchas.Tests/          (pruebas unitarias - MSTest/NUnit)
|   +-- UsuarioServiceTests.cs
|   +-- ReservaServiceTests.cs

```

Cada proyecto se nombra con el prefijo SistemaCanchas. seguido del nombre de la capa, y el namespace de cada archivo debe coincidir exactamente con el nombre del proyecto y su carpeta contenedora (namespace = ruta de carpetas). Regla de dependencia, según el diagrama A3: Presentación → Negocio → Datos, con Entidades como capa transversal referenciada por las tres anteriores; ningún proyecto de una capa inferior referencia a uno de una capa superior, ni se saltan capas (por ejemplo, un formulario jamás debe instanciar un Repository directamente).

### 3. Convenciones generales de nomenclatura

Se adopta el estándar oficial de Microsoft para C# (.NET Framework Design Guidelines) [1], con las adaptaciones específicas de este proyecto que se detallan en la sección 4.

| Elemento                       | Convención                      | Ejemplo                                  |
|--------------------------------|---------------------------------|------------------------------------------|
| Clases, records, structs       | PascalCase                      | ReservaService, ClienteRepository        |
| Interfaces                     | PascalCase con prefijo I        | IReservaService                          |
| Métodos                        | PascalCase (verbo + sustantivo) | RegistrarCliente(), ValidarConflicto()   |
| Propiedades públicas           | PascalCase                      | public string Nombre { get; set; }       |
| Parámetros y variables locales | camelCase                       | idCliente, fechaReserva                  |
| Campos privados                | camelCase con guion bajo        | _conexion, _idReserva                    |
| Constantes                     | PascalCase                      | const int DuracionTurnoMinutos = 60;     |
| Namespaces                     | PascalCase, igual a la carpeta  | SistemaCanchas.Negocio                   |
| Archivos .cs                   | = nombre de la clase pública    | ReservaService.cs                        |
| Enumeraciones (enum) y valores | PascalCase                      | enum EstadoReserva { Activa, Cancelada } |

**Regla general:** un solo tipo público (clase, interfaz o enum) por archivo. No se permiten archivos Utilidades.cs con múltiples clases sin relación.

## 4. Nomenclatura específica por capa

### 4.1 Capa de Presentación (Windows Forms)

Los formularios siguen el patrón definido en el artefacto A3 y deben mantenerse exactamente así para no romper la trazabilidad con el diagrama de arquitectura:

- Formularios: prefijo Frm + nombre de la entidad o función → FrmLogin, FrmClientes, FrmReservas, FrmPagos, FrmCanchas, FrmUsuarios.
- Controles de usuario reutilizables (UserControl): prefijo Ucl → UclGrillaHorarios.

Los controles individuales dentro de un formulario usan un prefijo de tres letras en minúscula (notación húngara reducida):

| Control          | Prefijo   | Ejemplo                   |
|------------------|-----------|---------------------------|
| Button           | btn       | btnRegistrar, btnCancelar |
| Label            | lbl       | lblEstadoPago             |
| TextBox          | txt       | txtNombre, txtCedula      |
| DataGridView     | dgv       | dgvClientes, dgvReservas  |
| DateTimePicker   | dtp       | dtpFecha                  |
| ComboBox         | cbo       | cboEstado, cboCancha      |
| CheckBox         | chk       | chkActivo                 |
| Panel / GroupBox | pnl / grp | pnlDatos, grpFiltros      |
| ErrorProvider    | err       | errValidacion             |

### 4.2 Capa de Negocio

- Clases de servicio: sufijo Service → UsuarioService, ClienteService, ReservaService, PagoService, CanchaService (ya definidas en A3).
- Cada servicio implementa su interfaz correspondiente (IUsuarioService, ...) para permitir pruebas unitarias con dobles de prueba (mocks).
- Excepciones de negocio propias: sufijo Exception, heredan de Exception (no de ApplicationException) → ReservaConflictoException, CredencialesInvalidasException.
- Métodos con el mismo nombre que en el diagrama A3: RegistrarCliente(), CrearReserva(), ValidarCredenciales(), ObtenerDisponibilidad(), etc.

### 4.3 Capa de Datos

- Clases de acceso a datos: sufijo Repository → UsuarioRepository, ClienteRepository, ReservaRepository, PagoRepository, CanchaRepository.
- Clase utilitaria compartida: GestorConexion, responsable de abrir la conexión con login\_bootstrap, invocar sp\_ObtenerCredencialesLogin y reabrir la conexión real con el login individual (usuario\_bd), conforme al flujo descrito en A1 §2.4 y en A3.

Los métodos CRUD se nombran de forma estandarizada en todas las clases Repository para que el equipo sepa qué esperar sin leer la implementación:

| Operación                     | Nombre del método                       | Retorno           |
|-------------------------------|-----------------------------------------|-------------------|
| Crear                         | Insertar(entidad)                       | int (Id generado) |
| Leer uno                      | Buscar(id)                              | Entidad o null    |
| Leer todos / filtrados        | ObtenerTodos() / ObtenerPorFecha(fecha) | List<Entidad>     |
| Actualizar                    | Actualizar(entidad)                     | bool              |
| Eliminar / dar de baja lógica | Eliminar(id) / Cancelar(id)             | bool              |

### 4.4 Capa de Entidades (POCOs)

Clases planas en singular que representan una fila de la base de datos, coherentes con el modelo lógico (A6): Cliente, Reserva, Horario, Pago, Cancha, Usuario, Rol. No contienen lógica de negocio ni acceso a datos, solo propiedades automáticas.

```
namespace SistemaCanchas.Entidades
{
    public class Usuario
    {
        public int IdUsuario { get; set; }
        public string NombreUsuario { get; set; }
        public string UsuarioLogin { get; set; }
        public string ClaveAppHash { get; set; }
        public string UsuarioBd { get; set; }
        public string ClaveBdEnc { get; set; }
        public int IdRol { get; set; }
        public string EstadoUsuario { get; set; }
    }
}
```

## 5. Convenciones de base de datos (SQL Server)

Esta sección documenta el estándar ya aplicado en el artefacto A11 (Script DDL), consistente con los modelos A6 (Modelo Lógico) y A7 (Modelo Físico).

| Elemento | Convención                       | Ejemplo                      |
|----------|----------------------------------|------------------------------|
| Tablas   | MAYÚSCULAS, sustantivo en plural | CLIENTES, RESERVAS, USUARIOS |

| Elemento                   | Convención                            | Ejemplo                                     |
|----------------------------|---------------------------------------|---------------------------------------------|
| Columnas                   | snake_case                            | id_cliente, fecha_pago, hora_inicio_horario |
| Llave primaria (PK)        | id_ <entidad en singular>             | id_cliente (PK de CLIENTES)                 |
| Llave foránea (FK)         | igual al nombre de la PK referenciada | id_cliente (FK en RESERVAS)                 |
| Restricción PRIMARY KEY    | PK_ <Tabla>                           | PK_CLIENTES                                 |
| Restricción FOREIGN KEY    | FK_ <TablaOrigen>_ <TablaDestino>     | FK_RESERVAS_CLIENTES                        |
| Restricción UNIQUE         | UQ_ <Tabla>_ <columna>                | UQ_CLIENTES_documento                       |
| Restricción CHECK          | CK_ <Tabla>_ <columna>                | CK_PAGOS_monto                              |
| Procedimientos almacenados | sp_ <Accion><Entidad>                 | sp_RegistrarCliente, sp_CrearReserva        |

### Consistencia obligatoria

Estos nombres deben coincidir exactamente entre el DDL (A11), las clases de Entidades y los parámetros de tipo SqlParameter en la capa de Datos. Un cambio de nombre de columna en el DDL implica actualizar las tres capas.

## 6. Estilo y formato del código C#

| Regla                   | Detalle                                                                                            |
|-------------------------|----------------------------------------------------------------------------------------------------|
| Llaves                  | En línea nueva, estilo Allman (predeterminado de Visual Studio).                                   |
| Indentación             | 4 espacios; nunca tabulaciones.                                                                    |
| Longitud de línea       | 120 caracteres como máximo sugerido.                                                               |
| using                   | Uno por línea; System.* primero, luego librerías de terceros, luego proyectos propios del sistema. |
| Modificadores de acceso | Siempre explícitos (private, public, internal); nunca depender del valor por defecto.              |
| var                     | Solo cuando el tipo es evidente por el lado derecho: var lista = new List<Cliente>();              |
| Un tipo por archivo     | Una clase, interfaz o enum pública por archivo .cs.                                                |

## 7. Comentarios y documentación en código

- Todo método público de las clases \*Service y \*Repository debe llevar comentario de documentación XML (///) con resumen, parámetros y valor de retorno.
- Los comentarios explican el porqué de una decisión, no repiten lo que el código ya dice.

- Cuando una validación implementa una regla de negocio del artefacto A1, se referencia explícitamente su código (RN01–RN13) para trazabilidad.

```

/// <summary>
/// Registra una nueva reserva validando disponibilidad de horario
/// y estado activo del cliente y del usuario que la registra.
/// </summary>
/// <param name="reserva">Datos de la reserva a crear.</param>
/// <returns>Id de la reserva creada.</returns>
/// <exception cref="ReservaConflictoException">
/// Se lanza si el horario solicitado ya esta ocupado.
/// </exception>
public int CrearReserva(Reserva reserva)
{
    // RN10: una cancha inactiva no puede recibir nuevas reservas
    if (!_canchaService.CanchaActiva(reserva.IdCancha))
        throw new InvalidOperationException("La cancha no esta activa.");

    // RN13: toda reserva registra el usuario que la crea (auditoria)
    reserva.IdUsuario = _sesionActual.IdUsuario;

    return _reservaRepository.Insertar(reserva);
}

```

## 8. Manejo de excepciones

- Prohibido el catch vacío (catch { }); toda excepción capturada se registra (log) o se relanza con contexto adicional.
- Capturar siempre el tipo de excepción más específico posible antes que la genérica Exception.
- La capa de Datos nunca muestra MessageBox.Show() ni ningún elemento de interfaz: solo lanza o propaga excepciones.
- Solo la capa de Presentación (los formularios) captura la excepción final y decide cómo mostrarla al usuario.
- Toda conexión SQL se abre dentro de un bloque using para garantizar su liberación incluso si ocurre una excepción.

## 9. Estándares de acceso a datos con ADO.NET

El artefacto A11 implementa toda la lógica de acceso mediante procedimientos almacenados (sp\_\*). En consecuencia, la capa de Datos no construye sentencias SQL de forma dinámica: cada método de un Repository invoca un procedimiento almacenado con CommandType.StoredProcedure y parámetros tipados.

- Prohibido concatenar valores directamente en sentencias SQL o construir texto SQL dinámico desde C#. Todo acceso pasa por un procedimiento almacenado existente en A11, invocado con SqlParameter.
- La cadena de conexión inicial (login\_bootstrap) se centraliza en GestorConexion y se lee desde App.config (<connectionStrings>), protegida con DPAPI conforme a RNF12; nunca se escribe literal (hardcoded) dentro de una clase Repository.

- Uso de SqlDataAdapter + DataTable cuando el resultado alimenta directamente un DataGridView; uso de SqlDataReader para lecturas secuenciales; ExecuteScalar para valores únicos (conteos, existencia).
- Cada método de un Repository abre y cierra su propia conexión (bloque using); no se comparten conexiones abiertas entre métodos.

```
public bool RegistrarCliente(Cliente cliente)
{
    using (SqlConnection conexion = _gestorConexion.ObtenerConexionActiva())
    using (SqlCommand cmd = new SqlCommand("sp_RegistrarCliente", conexion))
    {
        cmd.CommandType = CommandType.StoredProcedure;
        cmd.Parameters.AddWithValue("@nombre_cliente", cliente.Nombre);
        cmd.Parameters.AddWithValue("@tipo_documento_cliente", cliente.TipoDocumento);
        cmd.Parameters.AddWithValue("@numero_documento_cliente", cliente.NumeroDocumento);
        try
        {
            conexion.Open();
            return cmd.ExecuteNonQuery() > 0;
        }
        catch (SqlException ex)
        {
            throw new ApplicationException("Error al registrar el cliente.", ex);
        }
    }
}
```

## 10. Credenciales de doble capa: hash de aplicación y cifrado del login de motor

Conforme a A1 §2.4 y al modelo físico (A7), cada persona posee dos credenciales distintas gestionadas por la tabla USUARIOS: una credencial de aplicación (usuario\_login / clave\_app\_hash) y el login real de SQL Server (usuario\_bd / clave\_bd\_enc). Esta sección fija las librerías y decisiones de implementación que todo el equipo debe usar de forma uniforme, para que ningún módulo resuelva el cifrado de manera distinta al resto.

### 10.1 Hash de la credencial de aplicación (clave\_app\_hash)

Se usa BCrypt, mediante el paquete NuGet BCrypt.Net-Next [2], para generar y verificar el hash de la clave de aplicación. BCrypt es un hash de una sola vía (no reversible) con costo configurable, adecuado porque clave\_app\_hash nunca necesita descifrarse: solo se compara contra la clave ingresada en el login.

```
// Al registrar un usuario (UsuarioService.RegistrarUsuario)
string claveAppHash = BCrypt.Net.BCrypt.HashPassword(claveAppPlana, workFactor: 12);

// Al validar el login (UsuarioService.ValidarCredenciales)
bool esValida = BCrypt.Net.BCrypt.Verify(claveAppIngresada, usuario.ClaveAppHash);
```

### 10.2 Cifrado del login real de motor (clave\_bd\_enc)

A diferencia de clave\_app\_hash, la clave del login individual de SQL Server (usuario\_bd) debe poder recuperarse en texto plano para que GestorConexion reabra la conexión real; un hash no permite esto. Se usa cifrado simétrico reversible con el algoritmo AES (Advanced Encryption Standard, un cifrado de bloque



estándar de la industria) mediante la clase `System.Security.Cryptography.Aes` del propio .NET Framework [3], en modo CBC con un vector de inicialización (IV) aleatorio por cada valor cifrado, prefijado a la cadena resultante antes de convertirla a Base64 para su almacenamiento en `clave_bd_enc`.

Este cifrado es independiente del DPAPI (Data Protection API) que protege la cadena de conexión local en `App.config` conforme a RNF12: son dos mecanismos con propósitos distintos y no deben confundirse ni compartir clave. DPAPI protege un secreto que solo existe en el equipo local (la cadena de conexión), mientras que `clave_bd_enc` viaja dentro de la base de datos y debe poder descifrarse desde cualquier estación autorizada que ejecute la aplicación.

### 10.3 Ubicación de la clave de cifrado AES

La clave simétrica usada para cifrar y descifrar `clave_bd_enc` se almacena en `App.config`, dentro de una sección de configuración propia (no en `<connectionStrings>`), y se protege en reposo con DPAPI en ámbito `LocalMachine` (`ProtectedConfigurationProvider = DataProtectionConfigurationProvider` [4], con `useMachineProtection` habilitado). Se eligió el ámbito de máquina y no el de usuario porque el servicio puede ejecutarse bajo distintas cuentas de Windows en cada puesto de trabajo, y cualquier sesión autorizada en esa máquina debe poder descifrar la sección sin depender de un perfil de usuario específico.

#### Regla clave

La clave AES nunca se escribe en código fuente, nunca se guarda en texto plano en `App.config` ni en el DDL, y nunca se versiona en el repositorio. Solo `GestorConexion`, en la capa de Datos, tiene acceso a la sección de configuración protegida; ninguna otra clase la lee directamente.

Nota de coherencia entre artefactos: el comentario original del procedimiento `sp_RegistrarUsuario` en A11 indica que `clave_bd_enc` se cifra con DPAPI. Esta sección confirma que la decisión vigente del equipo es AES vía `System.Security.Cryptography`, conforme a lo ya redactado en A1 §2.4. El comentario de A11 queda pendiente de corrección para eliminar esa referencia a DPAPI y evitar la inconsistencia entre ambos artefactos.

## 11. Validaciones y reglas de negocio

Para mantener el desacoplamiento de capas del artefacto A3, las validaciones se reparten así:

| Capa                | Qué valida                                                                                                                        |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Presentación (Form) | Formato de entrada: campos vacíos, tipo de dato, longitud máxima de un <code>TextBox</code> , uso de <code>ErrorProvider</code> . |
| Negocio (Service)   | Reglas de dominio RN01–RN13: disponibilidad, conflictos de horario, estados válidos, credenciales, auditoría de reservas.         |
| Datos (Repository)  | Integridad a nivel de consulta (parámetros correctos); no implementa reglas de negocio.                                           |
| Base de Datos       | Integridad referencial y de dominio final: PK, FK, CHECK, NOT NULL, UNIQUE (última línea de defensa).                             |

**Regla clave**

Ninguna regla de negocio (RN01–RN13) se implementa únicamente en el formulario. El formulario puede replicar una validación por UX (por ejemplo, deshabilitar un botón), pero la validación autoritativa siempre vive en la capa de Negocio.

**12. Control de versiones (Git y GitHub)**

| Aspecto            | Convención                                                                                                 |
|--------------------|------------------------------------------------------------------------------------------------------------|
| Rama principal     | main — protegida, solo recibe cambios vía Pull Request.                                                    |
| Ramas de trabajo   | feature/<módulo>-<descripción-breve>, ej. feature/reservas-crear, feature/usuarios-login.                  |
| Mensajes de commit | En español, modo imperativo y breve: "Agrega validación de conflicto de horario en ReservaService".        |
| Pull Request       | Obligatorio antes de fusionar a main; requiere al menos 1 revisor del equipo.                              |
| Archivos ignorados | bin/, obj/, .vs/, *.user — incluir el .gitignore estándar de Visual Studio desde el primer commit de /src. |

**13. Checklist de revisión de código (antes de un Pull Request)**

1. ¿Los nombres de clases, métodos y controles siguen las convenciones de las secciones 3 y 4?
2. ¿El código respeta el sentido único de dependencia entre capas (Presentación → Negocio → Datos)?
3. ¿Toda invocación a la base de datos usa un procedimiento almacenado existente en A11, con SqlParameter y CommandType.StoredProcedure?
4. ¿Toda SqlConnection está dentro de un bloque using?
5. ¿Se evitan los catch vacíos y las excepciones genéricas innecesarias?
6. ¿Los métodos públicos de Service y Repository tienen comentario XML de documentación?
7. ¿Las reglas de negocio referenciadas (RNxx) están comentadas en el punto donde se validan?
8. ¿El nombre de tablas/columnas usado coincide con el definido en el DDL (A11)?
9. ¿clave\_app\_hash se genera y valida con BCrypt, y clave\_bd\_enc con AES, sin mezclar ambos mecanismos?
10. ¿El mensaje de commit es claro y sigue la convención de la sección 12?

## 14. Referencias

- [1] Microsoft. (2023). Common C# code conventions. Microsoft Learn. Disponible en acceso libre: <https://learn.microsoft.com/dotnet/csharp/fundamentals/coding-style/coding-conventions>
- [2] BCrypt.Net-Next (v4.0.3). (2023). Paquete NuGet — implementación .NET del algoritmo bcrypt de Provos y Mazières (1999). Acceso libre (licencia MIT): <https://www.nuget.org/packages/BCrypt.Net-Next>
- [3] Microsoft. (2023). Aes Class — System.Security.Cryptography. .NET API Browser. Documentación oficial de referencia para AES en .NET Framework 4.8. Acceso libre: <https://learn.microsoft.com/dotnet/api/system.security.cryptography.aes>
- [4] Microsoft. (2023). DataProtectionConfigurationProvider Class. .NET API Browser. Documentación oficial del proveedor de protección de configuración basado en DPAPI. Acceso libre: <https://learn.microsoft.com/en-us/dotnet/api/system.configuration.dpapiprotectedconfigurationprovider>
- [5] Pressman, R. S., & Maxim, B. R. (2014). Software Engineering: A Practitioner's Approach (8.<sup>a</sup> ed.). McGraw-Hill. ISBN 978-0-07-802212-8
- [6] Sommerville, I. (2016). Software Engineering (10.<sup>a</sup> ed.). Pearson. ISBN 978-0-13-394303-0